PRODUCT ASSIGNMNET-DSA[TREE'S]

UDAY CHIRRA
2303A510G2

1. Delete a Node from a Binary Search Tree

```
class Main {
   static class Node {
      int data;
      Node left, right;

      Node(int data) {
         this.data = data;
      }
   }

   static Node insert(Node root, int data) {
      if (root == null)
         return new Node(data);

      if (data < root.data)
         root.left = insert(root.left, data);
      else
         root.right = insert(root.right, data);

      return root;
   }

   static Node delete(Node root, int key) {
      if (root == null)
         return null;

      if (key < root.data) {
         root.left = delete(root.left, key);
      } else if (key > root.data) {
         root.right = delete(root.right, key);
      } else {
         if (root.left == null)
            return root.right;

         if (root.right == null)
            return root.left;

         Node successor = root.right;
```

```
            while (successor.left != null)
                successor = successor.left;

            root.data = successor.data;
            root.right = delete(root.right, successor.data);
        }

        return root;
    }

    static void inorder(Node root) {
        if (root == null)
            return;

        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }

    public static void main(String[] args) {
        Node root = null;

        root = insert(root, 50);
        root = insert(root, 30);
        root = insert(root, 70);
        root = insert(root, 20);
        root = insert(root, 40);
        root = insert(root, 60);
        root = insert(root, 80);

        root = delete(root, 50);

        inorder(root);
    }
}
```

2. Check if a Binary Tree is a Binary Search Tree

```
class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
```

```
    }
  }

  static boolean isBST(Node root, long min, long max) {
    if (root == null)
      return true;

    if (root.data <= min || root.data >= max)
      return false;

    return isBST(root.left, min, root.data) &&
        isBST(root.right, root.data, max);
  }

  public static void main(String[] args) {
    Node root = new Node(50);
    root.left = new Node(30);
    root.right = new Node(70);
    root.left.left = new Node(20);
    root.left.right = new Node(40);
    root.right.left = new Node(60);
    root.right.right = new Node(80);

    if (isBST(root, Long.MIN_VALUE, Long.MAX_VALUE))
      System.out.println("BST");
    else
      System.out.println("Not a BST");
  }
}
```

3. Find kth Smallest and kth Largest Elements in a BST

```
class Main {
  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }
```

```
static int count = 0;
static int result = -1;

static void kthSmallest(Node root, int k) {
    if (root == null)
        return;

    kthSmallest(root.left, k);

    count++;

    if (count == k) {
        result = root.data;
        return;
    }

    kthSmallest(root.right, k);
}

static void kthLargest(Node root, int k) {
    if (root == null)
        return;

    kthLargest(root.right, k);

    count++;

    if (count == k) {
        result = root.data;
        return;
    }

    kthLargest(root.left, k);
}

public static void main(String[] args) {
    Node root = new Node(50);
    root.left = new Node(30);
    root.right = new Node(70);
    root.left.left = new Node(20);
    root.left.right = new Node(40);
    root.right.left = new Node(60);
    root.right.right = new Node(80);
```

```
        int k = 3;

        count = 0;
        result = -1;
        kthSmallest(root, k);
        System.out.println("Kth Smallest: " + result);

        count = 0;
        result = -1;
        kthLargest(root, k);
        System.out.println("Kth Largest: " + result);
    }
}
```

4. Check Whether BST is Balanced or Not

```
class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }

    static int height(Node root) {
        if (root == null)
            return 0;

        int left = height(root.left);

        if (left == -1)
            return -1;

        int right = height(root.right);

        if (right == -1)
            return -1;

        if (Math.abs(left - right) > 1)
            return -1;
```

```
        return Math.max(left, right) + 1;
    }

    static boolean isBalanced(Node root) {
        return height(root) != -1;
    }

    public static void main(String[] args) {
        Node root = new Node(50);
        root.left = new Node(30);
        root.right = new Node(70);
        root.left.left = new Node(20);
        root.left.right = new Node(40);
        root.right.left = new Node(60);
        root.right.right = new Node(80);

        if (isBalanced(root))
            System.out.println("Balanced");
        else
            System.out.println("Not Balanced");
    }
}
```